

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống web quản lý và giám sát
ROV tích hợp trí tuệ nhân tạo

Lê Minh Thành

20222574

Chương trình đào tạo: Công nghệ Thông tin

Giảng viên hướng dẫn: [Tên giảng viên hướng dẫn]

Khoa: Khoa học Máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN
THÔNG

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống web quản lý và giám sát
ROV tích hợp trí tuệ nhân tạo

Sinh viên thực hiện: Lê Minh Thành

Mã số sinh viên: 20222574

Lớp: [Mã lớp]

Khoá: 67

Chương trình đào tạo: Công nghệ Thông tin

Giảng viên hướng dẫn: [Học hàm học vị. Họ và tên GVHD]

Khoa: Khoa học Máy tính

Trường: Công nghệ Thông tin và Truyền thông

LỜI CẢM ƠN

Trước tiên, em xin bày tỏ lòng biết ơn sâu sắc đến [Học hàm học vị. Tên GVHD], giảng viên hướng dẫn đồ án tốt nghiệp của em. Thầy/Cô đã tận tình hướng dẫn, định hướng giải pháp kỹ thuật và góp ý nội dung xuyên suốt quá trình thực hiện đề tài. Những nhận xét chi tiết và yêu cầu khắt khe của Thầy/Cô là động lực để em không ngừng cải thiện chất lượng của sản phẩm và báo cáo.

Em xin chân thành cảm ơn các thầy cô trong Khoa Khoa học Máy tính, Trường Công nghệ Thông tin và Truyền thông, Đại học Bách Khoa Hà Nội đã truyền đạt kiến thức nền tảng vững chắc trong suốt bốn năm học. Những kiến thức về lập trình, cơ sở dữ liệu, mạng máy tính và kỹ thuật phần mềm là nền tảng thiết yếu để em thực hiện đề tài này.

Em cũng xin gửi lời cảm ơn đến các bạn trong nhóm học tập đã cùng thảo luận, chia sẻ kinh nghiệm và hỗ trợ nhau trong quá trình nghiên cứu. Môi trường học tập tích cực và tinh thần hỗ trợ lẫn nhau đã giúp em vượt qua nhiều thách thức kỹ thuật.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc đến gia đình đã luôn động viên, tạo điều kiện tốt nhất để em yên tâm học tập và hoàn thành đồ án tốt nghiệp.

Do kiến thức và kinh nghiệm còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy cô và các bạn để đồ án được hoàn thiện hơn.

Hà Nội, tháng 6 năm 2026

Sinh viên thực hiện

Lê Minh Thành

TÓM TẮT NỘI DUNG ĐỒ ÁN

Tên đề tài: Xây dựng hệ thống web quản lý và giám sát ROV tích hợp trí tuệ nhân tạo

Sinh viên: Lê Minh Thành **MSSV:** 20222574

Giảng viên hướng dẫn: [Học hàm học vị. Tên GVHD]

Tóm tắt:

Đồ án tốt nghiệp này trình bày quá trình thiết kế và xây dựng một hệ thống web toàn diện phục vụ quản lý và giám sát hoạt động của ROV (Remotely Operated Vehicle — phương tiện vận hành từ xa dưới nước). Hệ thống được phát triển nhằm giải quyết bài toán thực tế: sau mỗi chuyến hoạt động ngoài khơi, operator cần một nền tảng tập trung để upload, lưu trữ và phân tích dữ liệu sensor, video, ảnh thu thập được từ ROV.

Hệ thống được xây dựng theo kiến trúc ba tầng với frontend là ứng dụng React 18 SPA, backend là REST API Node.js/Express, và tầng dữ liệu bao gồm MongoDB Atlas (dữ liệu cấu trúc), AWS S3 (media) và Redis (cache/queue). Ngoài ra, một microservice Python FastAPI tách biệt được triển khai để chạy mô hình nhận diện vật thể YOLOv8, xử lý bất đồng bộ qua Bull Queue.

Các tính năng chính bao gồm: (1) Quản lý fleet ROV, trip và dive với phân quyền RBAC ba cấp (admin/operator/viewer); (2) Upload và visualize dữ liệu sensor từ file CSV với phát hiện bất thường tự động bằng Z-Score; (3) Quản lý media với pipeline phân tích YOLOv8 bất đồng bộ và hệ thống Evidence cho phép chụp snapshot và đánh dấu clip video; (4) Đồng bộ hóa sensor data với video theo thời gian thực (Sensor-Video Sync) thông qua trường timestamp; (5) Sinh tóm tắt trip tự động bằng Google Gemini 2.5 Flash; (6) Thông báo realtime qua Server-Sent Events.

Sản phẩm đạt được là ứng dụng web hoàn chỉnh, responsive, hỗ trợ dark mode, có thể triển khai với Docker Compose trên VPS. Frontend được deploy trên Vercel. Hệ thống đã được kiểm thử với dữ liệu thực và đáp ứng đầy đủ các yêu cầu đề ra.

Từ khoá: ROV, quản lý dữ liệu, React, Node.js, MongoDB, YOLOv8, Gemini AI, phát hiện bất thường, Z-Score

ABSTRACT

Title: Design and Development of an AI-Integrated Web Platform for ROV Management and Monitoring

Student: Le Minh Thanh **Student ID:** 20222574

Supervisor: [Supervisor's title and name]

Abstract:

This thesis presents the design and implementation of a comprehensive web platform for managing and monitoring the operations of Remotely Operated Vehicles (ROVs). The system addresses a practical operational need: after each offshore mission, operators require a centralized platform to upload, store, and analyze the sensor data, video footage, and images collected by ROVs.

The system follows a three-tier architecture with a React 18 SPA frontend, a Node.js/Express REST API backend, and a data layer comprising MongoDB Atlas (structured data), AWS S3 (media files), and Redis (caching/queue). A separate Python FastAPI microservice handles YOLOv8-based object detection, with jobs processed asynchronously via Bull Queue.

Key features include: (1) Fleet management for ROVs, trips, and dives with three-level RBAC (admin/operator/viewer); (2) Sensor data upload and visualization from CSV files with automatic anomaly detection using the Z-Score algorithm; (3) Media management with an asynchronous YOLOv8 analysis pipeline and an Evidence System enabling video snapshot capture and clip bookmarking; (4) Real-time sensor-video synchronization via recorded timestamps and moving reference lines on charts; (5) Automated trip summary generation using Google Gemini 2.5 Flash; (6) Real-time push notifications via Server-Sent Events.

The deliverable is a fully functional, responsive, dark-mode-capable web application deployable via Docker Compose. The frontend is hosted on Vercel. The system has been tested with real-world data and meets all specified requirements.

Keywords: ROV, data management, React, Node.js, MongoDB, YOLOv8, Gemini AI, anomaly detection, Z-Score, sensor-video sync

MỤC LỤC

DANH MỤC HÌNH VẼ

DANH MỤC BẢNG BIỂU

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này giới thiệu bối cảnh và động lực thúc đẩy đề tài, trình bày mục tiêu cùng phạm vi nghiên cứu, định hướng giải pháp kỹ thuật và bố cục của báo cáo.

1.1 Đặt vấn đề

ROV (Remotely Operated Vehicle — phương tiện vận hành từ xa) là thiết bị lặn không người lái được điều khiển qua cáp từ tàu mẹ hoặc trạm điều hành trên bờ. Với khả năng hoạt động ở độ sâu hàng trăm đến hàng nghìn mét mà không đặt con người vào nguy hiểm, ROV được ứng dụng rộng rãi trong nhiều lĩnh vực: kiểm tra và bảo dưỡng đường ống dầu khí, cáp quang biển và móng nền công trình ngầm; khảo sát địa chất và sinh thái học biển; tìm kiếm cứu nạn; và hỗ trợ nghiên cứu khoa học dưới nước.

Trong mỗi chuyến hoạt động (gọi là *trip*), ROV thực hiện nhiều lần lặn (gọi là *dive*). Trong suốt quá trình lặn, hệ thống cảm biến trên ROV liên tục ghi lại nhiều thông số: độ sâu, nhiệt độ nước, áp suất, tọa độ GPS, điện áp và dung lượng pin, độ ẩm buồng điện tử, cũng như dữ liệu định hướng góc quay (yaw, pitch, roll). Song song đó, các camera lắp trên ROV ghi lại video độ phân giải cao và chụp ảnh định kỳ hoặc theo yêu cầu của operator.

Một đặc thù quan trọng của môi trường vận hành ROV là hoạt động ở vùng biển xa bờ, nơi không có kết nối internet đủ tin cậy. Điều này dẫn đến quy trình *post-mission data offloading*: toàn bộ dữ liệu cảm biến, video và ảnh chỉ được chuyển về hệ thống lưu trữ sau khi tàu cập bến. Một chuyến khảo sát điển hình kéo dài từ vài ngày đến vài tuần có thể tạo ra hàng chục gigabyte dữ liệu từ nhiều lần lặn của một hoặc nhiều ROV.

Thực trạng quản lý dữ liệu post-mission tại nhiều đơn vị vận hành ROV hiện nay còn tồn tại nhiều hạn chế đáng kể:

1. **Dữ liệu phân tán, thiếu hệ thống tập trung:** Dữ liệu cảm biến thường được lưu dưới dạng file CSV/JSON rời rạc trên nhiều máy tính khác nhau. Không có công cụ thống nhất để so sánh dữ liệu giữa các lần lặn hay theo dõi xu hướng theo thời gian.
2. **Quản lý media thủ công:** Video và ảnh được lưu trên ổ cứng cục bộ, thiếu hệ thống phân loại, tìm kiếm và xem lại hiệu quả. Việc phân tích thủ công để tìm vật thể bất thường trong hàng giờ video là công việc tốn nhiều nhân lực.

3. **Thiếu công cụ quản lý fleet:** Khi vận hành nhiều ROV trên nhiều chuyến song song, không có hệ thống tập trung để theo dõi trạng thái thiết bị, lịch sử bảo trì hay lịch sử sử dụng.
4. **Báo cáo và phân tích thủ công:** Việc tổng hợp kết quả sau mỗi chuyến lặn được thực hiện thủ công bởi kỹ sư, tốn nhiều thời gian và dễ xảy ra thiếu sót, không nhất quán.

Những hạn chế trên đặt ra nhu cầu cấp thiết về một hệ thống web tập trung, cho phép operator upload dữ liệu sau khi về bờ, quản lý fleet ROV, phân tích dữ liệu lịch sử và tận dụng các công nghệ trí tuệ nhân tạo hiện đại để tự động hóa một phần công việc phân tích.

1.2 Mục tiêu và phạm vi đề tài

Trước khi xác định mục tiêu, cần đánh giá các sản phẩm và giải pháp hiện có cho bài toán quản lý ROV.

QGroundControl (QGC) qgc2024 là phần mềm GCS (Ground Control Station) mã nguồn mở phổ biến nhất, hỗ trợ điều khiển drone và ROV realtime qua giao thức MAVLink. Tuy nhiên, đây là ứng dụng desktop, tập trung hoàn toàn vào điều khiển trong thời gian thực, không có web interface, không hỗ trợ quản lý fleet hay phân tích dữ liệu lịch sử post-mission.

BlueOS blueos2024 của Blue Robotics là companion computer software chạy trực tiếp trên ROV, cung cấp web interface để cấu hình và giám sát trạng thái thiết bị. Phạm vi BlueOS giới hạn ở việc quản lý một ROV đơn lẻ đang hoạt động; không hỗ trợ quản lý đa thiết bị hay lưu trữ và phân tích dữ liệu sau chuyến lặn.

Các hệ thống thương mại như VideoRay Cockpit hay ECA ROVStation có nhiều tính năng hơn nhưng chi phí bản quyền cao, không linh hoạt trong tùy biến và không tích hợp các công nghệ AI thế hệ mới.

Từ phân tích trên, đề tài xác định mục tiêu xây dựng hệ thống web quản lý ROV toàn diện với các chức năng chính sau:

- Quản lý fleet ROV: đăng ký thiết bị, theo dõi trạng thái hoạt động, xem lịch sử trip.
- Quản lý trip và dive: tổ chức dữ liệu theo cấu trúc phân cấp trip → dive, hỗ trợ filter và search nâng cao.
- Upload và phân tích dữ liệu cảm biến: nhận file CSV/JSON, visualize trên biểu đồ, phát hiện bất thường tự động.

- Quản lý media trên cloud: upload video/ảnh lên AWS S3, gallery với drag-to-reorder, xem lại trong trình phát tích hợp.
- Tích hợp AI: nhận diện vật thể trong ảnh/video (YOLOv8), tổng hợp báo cáo trip tự động (Gemini 2.5 Flash).
- Phân quyền RBAC ba cấp: admin, operator, viewer.
- Giao diện web responsive, hỗ trợ dark/light mode.

Phạm vi ngoài đề tài: Hệ thống không hỗ trợ điều khiển ROV realtime, không xử lý luồng dữ liệu trực tiếp khi ROV đang lặn. Toàn bộ dữ liệu được upload thủ công bởi operator sau khi về bờ.

1.3 Định hướng giải pháp

Từ yêu cầu đã xác định, đề tài tiếp cận giải pháp theo hướng xây dựng ứng dụng web hiện đại với kiến trúc phân tầng rõ ràng và các công nghệ được lựa chọn kỹ lưỡng.

Frontend là ứng dụng SPA (Single Page Application) xây dựng bằng React 18 **react2024** với Vite **vite2024**, cho phép trải nghiệm người dùng mượt mà không tải lại trang. Tailwind CSS **tailwind2024** được chọn để xây dựng design system nhất quán với hỗ trợ dark mode. React Query và Zustand quản lý state phía client.

Backend là REST API viết bằng Node.js và Express **expressjs2024**, cung cấp tập hợp endpoint theo chuẩn RESTful. Tính năng realtime được triển khai qua SSE (Server-Sent Events) **sse_spec** — giao thức đơn giản hơn WebSocket, phù hợp với mô hình push notification một chiều từ server về client.

Tầng dữ liệu bao gồm MongoDB Atlas **mongodb2024** cho dữ liệu cấu trúc linh hoạt (sensor data có số trường thay đổi theo cấu hình ROV), AWS S3 **awss32024** cho media với presigned URL để tránh server làm proxy cho file lớn, và Redis **redis2024** phục vụ cả token blacklist lẫn message queue thông qua Bull.

Tầng AI bao gồm microservice Python FastAPI **fastapi2024** tách biệt chạy mô hình YOLOv8 **jocher2023yolov8** để nhận diện vật thể trong ảnh và video, và Google Gemini 2.5 Flash **gemini2024** để sinh tóm tắt trip. Các tác vụ AI nặng được xử lý bất đồng bộ qua Bull Queue để không block HTTP request.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát hiện trạng và phân tích yêu cầu hệ thống. Nội dung bao gồm đánh giá chi tiết các giải pháp hiện có, mô hình hóa chức năng qua biểu đồ use case tổng quát và phân rã, đặc tả chi tiết sáu use case quan trọng nhất và các yêu cầu phi chức năng về bảo mật, hiệu năng, khả năng mở rộng.

Chương 3 giới thiệu các công nghệ và nền tảng lý thuyết được sử dụng trong đề án. Với mỗi công nghệ, chương trình bày nguyên lý hoạt động, phân tích so sánh với các lựa chọn thay thế và giải thích lý do lựa chọn dựa trên yêu cầu cụ thể của hệ thống.

Chương 4 trình bày thiết kế kiến trúc ba tầng có tích hợp microservice AI, thiết kế cơ sở dữ liệu MongoDB, thiết kế API RESTful, thiết kế giao diện người dùng, quy trình xây dựng và minh họa ứng dụng, kết quả kiểm thử và mô hình triển khai với Docker Compose.

Chương 5 phân tích sâu năm giải pháp kỹ thuật đóng góp nổi bật của đề án, bao gồm cơ chế đồng bộ sensor-video, hệ thống phát hiện bất thường tự động, pipeline YOLO bất đồng bộ, hệ thống evidence và AI trip summary.

Chương 6 đưa ra kết luận toàn diện về kết quả đạt được so với mục tiêu đề ra, đánh giá những hạn chế còn tồn tại và định hướng phát triển trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này trình bày kết quả khảo sát các hệ thống hiện có, phân tích và mô hình hóa yêu cầu chức năng thông qua biểu đồ use case, đặc tả chi tiết các use case quan trọng, và các yêu cầu phi chức năng của hệ thống.

2.1 Khảo sát hiện trạng

Để xác định rõ khoảng trống mà đề tài cần lấp đầy, nhóm tiến hành khảo sát ba nhóm giải pháp chính đang được sử dụng trong cộng đồng ROV.

2.1.1 Các giải pháp hiện có

QGroundControl (QGC) qgc2024 là phần mềm GCS mã nguồn mở phổ biến nhất, được phát triển và duy trì bởi cộng đồng MAVLink. QGC hỗ trợ điều khiển realtime cho cả drone trên không và ROV dưới nước, cung cấp giao diện hiển thị trạng thái thiết bị, bản đồ và camera feed. Tuy nhiên, QGC là ứng dụng desktop (Windows/Mac/Linux/Android), không có web interface, không hỗ trợ quản lý fleet đa thiết bị, và không có tính năng lưu trữ hay phân tích dữ liệu post-mission.

BlueOS blueos2024 của Blue Robotics là companion computer software chạy trên ROV (thường trên Raspberry Pi), cung cấp web interface để cấu hình firmware, cài đặt extension và xem camera feed realtime. Phạm vi của BlueOS giới hạn ở việc cấu hình và giám sát một ROV đơn lẻ, không hỗ trợ quản lý multi-ROV, lưu trữ lịch sử dữ liệu hay phân tích sau chuyến lặn.

VideoRay Cockpit và các hệ thống thương mại tương tự là giải pháp toàn diện nhưng đi kèm chi phí bản quyền cao (thường từ vài nghìn đến hàng chục nghìn USD), không linh hoạt trong tùy biến theo quy trình riêng của từng đơn vị, và không tích hợp các công nghệ AI thế hệ mới như object detection hay text generation.

2.1.2 Bảng so sánh tổng quan

Bảng 2.1: So sánh các giải pháp quản lý ROV hiện có

Tiêu chí	QGC	BlueOS	Thương mại	Đề tài
Web interface	×	✓	✓	✓
Quản lý fleet (đa ROV)	×	×	✓	✓
Lưu trữ dữ liệu sensor	×	×	✓	✓
Visualize chart sensor	×	×	Giới hạn	✓
Phát hiện bất thường	×	×	×	✓
Quản lý media cloud	×	×	Giới hạn	✓
AI object detection	×	×	×	✓
AI trip summary	×	×	×	✓
Phân quyền RBAC	×	×	✓	✓
Mã nguồn mở / Miễn phí	✓	✓	×	✓

Bảng ?? cho thấy không có giải pháp hiện tại nào đáp ứng đầy đủ nhu cầu quản lý ROV post-mission với tích hợp AI. Đây là khoảng trống mà đề tài hướng đến lấp đầy.

2.2 Tổng quan chức năng

2.2.1 Các tác nhân hệ thống

Hệ thống phục vụ ba nhóm người dùng với quyền hạn khác nhau:

- **Admin:** Có toàn quyền trên hệ thống. Ngoài các quyền của Operator, Admin có thể quản lý tài khoản người dùng (tạo, khóa, đổi role), xóa bất kỳ resource nào, và xem audit log.
- **Operator:** Người vận hành ROV thực tế. Có thể tạo và chỉnh sửa ROV, trip, dive; upload dữ liệu sensor và media; kích hoạt phân tích AI; xuất báo cáo.
- **Viewer:** Người xem báo cáo (khách hàng, quản lý cấp cao). Chỉ có quyền đọc toàn bộ dữ liệu, không thể tạo hay sửa.

2.2.2 Biểu đồ use case tổng quát

[Hình 2.1: Biểu đồ use case tổng quát với 3 actor (Admin, Operator, Viewer) và các nhóm use case chính: Xác thực, Quản lý ROV, Quản lý Trip/Dive, Quản lý Sensor Data, Quản lý Media, Quản lý AI, Quản lý User, Audit Log]

Hình 2.1: Biểu đồ use case tổng quát hệ thống quản lý ROV

Hệ thống có sáu nhóm chức năng chính: (1) Xác thực và quản lý tài khoản, (2) Quản lý ROV, (3) Quản lý trip và dive, (4) Quản lý dữ liệu cảm biến, (5) Quản lý media và phân tích AI, (6) Audit log (admin only).

2.2.3 Biểu đồ use case phân rã — Quản lý dữ liệu cảm biến

[Hình 2.2: Biểu đồ use case phân rã Quản lý dữ liệu cảm biến: Upload CSV, Xem biểu đồ sensor, Xem anomaly, Đồng bộ video, Xóa data]

Hình 2.2: Biểu đồ use case phân rã — Quản lý dữ liệu cảm biến

2.2.4 Quy trình nghiệp vụ chính

Quy trình nghiệp vụ cốt lõi của hệ thống là *post-mission data processing*, bao gồm các bước tuần tự: Operator đăng nhập → Tạo Trip mới → Tạo Dive trong Trip → Upload sensor CSV cho Dive → Upload media (video/ảnh) cho Dive → Kích hoạt phân tích YOLO → Kích hoạt AI Trip Summary → Viewer

xem báo cáo. Quá trình này cho phép tổ chức toàn bộ dữ liệu một chuyển hoạt động theo cấu trúc phân cấp rõ ràng.

2.3 Đặc tả chức năng

2.3.1 UC01 — Đăng nhập hệ thống

Bảng 2.2: Đặc tả use case UC01 — Đăng nhập hệ thống

Tên use case	Đăng nhập hệ thống
Tác nhân	Tất cả người dùng
Tiền điều kiện	Người dùng đã có tài khoản trong hệ thống
Hậu điều kiện	Người dùng được cấp access token và redirect về dashboard
Luồng chính	1. Người dùng mở trang đăng nhập 2. Nhập email và mật khẩu, bấm “Đăng nhập” 3. Hệ thống xác thực thông tin 4. Hệ thống trả về access token (15 phút) và refresh token (7 ngày) 5. Người dùng được redirect về dashboard
Luồng phụ 1	Đăng nhập bằng Google OAuth2: Bấm “Continue with Google” → Chọn tài khoản → Callback xử lý token → Redirect dashboard
Luồng ngoại lệ	Email/mật khẩu sai → Hiện thị thông báo lỗi. Tài khoản bị khóa → Thông báo liên hệ admin

2.3.2 UC02 — Tạo và quản lý dive trong trip

Bảng 2.3: Đặc tả use case UC02 — Tạo và quản lý dive

Tên use case	Tạo và quản lý dive trong trip
Tác nhân	Operator, Admin
Tiền điều kiện	Trip đã tồn tại trong hệ thống
Hậu điều kiện	Dive mới được tạo và liên kết với trip, có thể nhận sensor data và media
Luồng chính	1. Operator mở trang chi tiết trip 2. Bấm “Add Dive”, điền thông tin: tên, thời gian, mô tả 3. Hệ thống tạo dive mới với trạng thái “pending” 4. Operator cập nhật trạng thái dive trong quá trình lặn (running → done/failed) 5. Hệ thống cập nhật badge trạng thái trong danh sách
Luồng ngoại lệ	Thiếu trường bắt buộc → Form validation hiển thị lỗi inline

2.3.3 UC03 — Upload dữ liệu cảm biến từ file CSV

Bảng 2.4: Đặc tả use case UC03 — Upload sensor data

Tên use case	Upload dữ liệu cảm biến từ file CSV
Tác nhân	Operator, Admin
Tiền điều kiện	Dive đã tồn tại. Operator có file CSV từ ROV
Hậu điều kiện	Readings được lưu vào DB. Dive cập nhật gpsLocation, locationName nếu CSV có lat/lng
Luồng chính	1. Trong trang chi tiết dive, operator chọn file CSV 2. Frontend parse và validate file 3. Gửi mảng readings lên POST /dives/:id/sensor-data/upload 4. Backend lưu bulk vào collection SensorData 5. Nếu CSV có lat/lng dòng đầu: gọi Nominatim reverse geocoding, lưu locationName 6. Hiển thị badge số readings mới
Luồng phụ	Upload lại (overwrite): Xóa data cũ trước khi lưu data mới
Luồng ngoại lệ	File thiếu cột bắt buộc (timestamp, depth, temp, pressure) → Thông báo lỗi cụ thể

2.3.4 UC04 — Upload media và kích hoạt phân tích YOLO

Bảng 2.5: Đặc tả use case UC04 — Upload và phân tích media

Tên use case	Upload media và phân tích YOLO
Tác nhân	Operator, Admin
Tiền điều kiện	Dive tồn tại. Operator có file video/ảnh cần upload
Hậu điều kiện	Media được lưu trên S3. Labels nhận diện vật thể được lưu sau khi YOLO xử lý xong
Luồng chính	1. Operator chọn file, điền metadata (mô tả, recordedAt tùy chọn) 2. Frontend yêu cầu presigned URL từ backend 3. Frontend upload trực tiếp lên S3 qua presigned URL 4. Frontend gọi confirm endpoint, media.status = “ready” 5. Operator bấm “Analyze” trong AIAalyzePopover: chọn model và confidence 6. Backend enqueue Bull job, set analysisStatus = “pending” 7. YOLO microservice xử lý, trả labels 8. SSE push thông báo “media_analysis_done” 9. Frontend tự động refetch media, hiển thị labels
Luồng ngoại lệ	YOLO service không phản hồi → Job retry 2 lần → analysisStatus = “failed”

2.3.5 UC05 — Xem chi tiết dive (cockpit view)

Bảng 2.6: Đặc tả use case UC05 — Xem chi tiết dive

Tên use case	Xem chi tiết dive
Tác nhân	Tất cả người dùng đã đăng nhập
Tiền điều kiện	Dive tồn tại và có dữ liệu sensor hoặc media
Hậu điều kiện	Không thay đổi trạng thái hệ thống
Luồng chính	1. Người dùng truy cập <code>/dives/:id</code> 2. Hệ thống tải layout cockpit 3 cột: trái (map + KPI), giữa (video player), phải (gauges + alerts) 3. Biểu đồ sensor hiển thị ở panel dưới với 3 tab (Environment, Navigation, System) 4. Nếu có media với <code>recordedAt</code>: khi video phát, <code>ReferenceLine</code> trên chart di chuyển theo timestamp 5. Người dùng có thể chuyển tab biểu đồ, toggle legend, zoom bằng Brush range selector
Luồng phụ	Xem evidence: click “Evidence N” → <code>EvidencePanel</code> → <code>EvidenceViewer</code> với bbox overlay

2.3.6 UC06 — Tổng hợp trip bằng AI

Bảng 2.7: Đặc tả use case UC06 — AI trip summary

Tên use case	Tổng hợp trip bằng AI
Tác nhân	Operator, Admin
Tiền điều kiện	Trip có trạng thái “completed”
Hậu điều kiện	aiSummary.content được lưu vào trip document, hiển thị cho tất cả người dùng
Luồng chính	1. Operator bấm “Generate Summary” trong trang chi tiết trip 2. Backend set aiSummary.status = “pending”, enqueue Bull job 3. Trả 202 Accepted ngay lập tức 4. Bull worker gọi Gemini API với prompt tổng hợp từ dữ liệu trip 5. Lưu kết quả vào aiSummary.content, set status = “done” 6. SSE push notification đến user 7. Frontend poll hoặc nhận SSE → hiển thị summary
Luồng ngoại lệ	Gemini API lỗi → Retry 2 lần → aiSummary.status = “failed”

2.4 Yêu cầu phi chức năng

2.4.1 Bảo mật

Hệ thống áp dụng mô hình xác thực JWT `jwt_rfc` hai tầng: access token ngắn hạn (15 phút) dùng để xác thực mỗi API request, và refresh token dài hạn (7 ngày) lưu trong HttpOnly cookie để cấp access token mới. Khi người dùng đăng xuất, refresh token bị xóa khỏi cơ sở dữ liệu và được thêm vào Redis blacklist với TTL 7 ngày, ngăn chặn việc tái sử dụng token cũ. Rate limiting được áp dụng trên các endpoint xác thực (20 request/15 phút/IP) để chống brute-force. Toàn bộ giao tiếp qua HTTPS trong môi trường production.

2.4.2 Hiệu năng

Các danh sách dữ liệu (trips, dives, ROVs, media) hỗ trợ phân trang server-side với page size mặc định là 20. Media không được route qua backend server mà được truy cập trực tiếp qua S3 presigned URL, giảm tải đáng kể băng thông server. Tác vụ AI nặng (YOLO, Gemini) chạy bất đồng bộ qua Bull Queue, không block HTTP request. MongoDB indexes được thêm cho các

trường truy vấn thường xuyên.

2.4.3 Tính nhất quán dữ liệu

Response format được chuẩn hóa toàn bộ qua hàm `success(res, data)` với cấu trúc `{success, message, data}`. Tất cả mutation operation đều có xác nhận (confirm dialog) trước khi thực thi. Bulk operations (xóa nhiều, đổi role hàng loạt) được thực hiện trong một transaction-like flow và có audit log tương ứng.

2.4.4 Khả năng mở rộng

YOLO microservice tách biệt hoàn toàn về mặt process, có thể scale độc lập hoặc thay thế bằng mô hình khác mà không ảnh hưởng backend chính. Hệ thống quản lý model AI qua metadata dictionary trong microservice, cho phép thêm mô hình fine-tuned mới chỉ bằng cách sao chép file `.pt` và thêm một entry vào dict, không cần deploy lại.

2.4.5 Khả năng sử dụng

Giao diện responsive hoạt động trên màn hình từ 375px (mobile) đến 1920px (desktop). Hỗ trợ dark mode và light mode với design token system nhất quán. Tất cả danh sách có skeleton loading state và empty state thân thiện. Tất cả form có inline validation real-time. Export CSV và PDF được hỗ trợ cho tất cả danh sách.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này giới thiệu các công nghệ và nền tảng lý thuyết được sử dụng trong đề tài. Với mỗi công nghệ, chương trình bày khái niệm, phân tích lựa chọn thay thế và giải thích lý do chọn dựa trên yêu cầu cụ thể đã xác định ở Chương 2.

3.1 Công nghệ frontend

3.1.1 React 18 và mô hình component

React **react2024** là thư viện JavaScript mã nguồn mở do Meta phát triển, được sử dụng để xây dựng giao diện người dùng dạng component. React 18 giới thiệu Concurrent Mode cho phép render ngắt quãng (interruptible rendering), cải thiện hiệu năng với các tác vụ render tốn kém. Kiến trúc Virtual DOM giúp React tính toán diff hiệu quả trước khi cập nhật DOM thực, giảm số lần re-render không cần thiết.

So sánh lựa chọn thay thế: Vue 3 cũng là lựa chọn hợp lệ với cú pháp đơn giản hơn và Composition API tương tự React hooks. Tuy nhiên, React được chọn vì: (1) Hệ sinh thái lớn hơn đáng kể với nhiều thư viện component như shadcn/ui **shadcn2024**; (2) Cộng đồng lớn hơn, dễ tìm giải pháp cho vấn đề đặc thù; (3) React Query, Zustand và các thư viện state management hàng đầu đều ưu tiên hỗ trợ React.

Angular không được xem xét vì overhead lớn (class-based, decorators, module system phức tạp) không phù hợp với đồ án quy mô vừa.

3.1.2 Vite — Build tool thế hệ mới

Vite **vite2024** là build tool và development server hiện đại sử dụng ES Modules (ESM) native của trình duyệt trong quá trình phát triển, loại bỏ bước bundling toàn bộ. Thời gian khởi động dev server của Vite điển hình dưới 1 giây, và Hot Module Replacement (HMR) cập nhật module trong thời gian dưới 100ms.

Lựa chọn thay thế chính là Create React App (CRA), nhưng CRA dùng webpack với thời gian khởi động và HMR chậm hơn đáng kể. Next.js là framework đầy đủ tính năng (SSR, file-based routing) nhưng thêm complexity không cần thiết cho hệ thống quản lý nội bộ không yêu cầu SEO.

3.1.3 Tailwind CSS và design token system

Tailwind CSS **tailwind2024** là CSS framework theo hướng utility-first, cung cấp các class nhỏ như `flex`, `gap-4`, `text-sm` thay vì component CSS có sẵn. Hệ thống có thể cấu hình design tokens (màu sắc, spacing, typography) trong file cấu hình, đảm bảo nhất quán thiết kế toàn ứng dụng.

So với Material UI (MUI) hay Ant Design: các thư viện component có sẵn nhanh hơn để bắt đầu nhưng khó override styles khi cần tùy biến sâu. Tailwind cho phép kiểm soát pixel-perfect mà không phải chiến đấu với CSS specificity. Dark mode với Tailwind đơn giản hơn: thêm class `dark:` prefix là đủ, nhờ cơ chế `darkMode: 'class'` trong config.

3.1.4 React Query và Zustand

React Query (TanStack Query) reactquery2024 là thư viện quản lý server state cho React. Thay vì quản lý loading/error/data bằng tay với `useState` và `useEffect`, React Query tự động handle caching, background refetch, stale-while-revalidate và query invalidation. Với `queryClient.invalidateQueries` toàn bộ cache liên quan được làm mới sau khi mutation — loại bỏ logic sync phức tạp.

Zustand zustand2024 là thư viện client state management nhỏ gọn (1KB). Được dùng cho auth state (user, tokens) và theme preference — những trạng thái không phải server state. So với Redux: Zustand không yêu cầu boilerplate (reducers, actions, selectors), phù hợp với client state đơn giản.

3.1.5 Recharts và Leaflet.js

Recharts recharts2024 là thư viện biểu đồ xây dựng hoàn toàn trên React và SVG, sử dụng mô hình component composable. Đặc điểm quan trọng: dễ tích hợp custom dots (để highlight anomaly), custom Reference Lines (để vẽ đường sync video), và Legend clickable để toggle metric. Chart.js không cơ bản là React-native, gây khó khăn khi cần deep integration với React state.

Leaflet.js leaflet2024 kết hợp với OpenStreetMap là giải pháp bản đồ miễn phí không cần API key. React-Leaflet (wrapper React chính thức cho Leaflet) không tương thích với React 18.3 do lifecycle issues, vì vậy Leaflet raw được sử dụng trực tiếp qua `useEffect` để khởi tạo map instance.

3.2 Công nghệ backend

3.2.1 Node.js và Express

Node.js nodejs2024 là JavaScript runtime xây dựng trên V8 engine của Chrome, sử dụng mô hình I/O bất đồng bộ không chặn (non-blocking I/O) dựa trên event loop. Đặc điểm này làm Node.js hiệu quả với các tác vụ I/O-bound như đọc/ghi database, gọi API bên ngoài — phù hợp với backend RESTful xử lý nhiều concurrent request.

Lựa chọn thay thế: Django (Python) có nhiều tính năng built-in hơn nhưng synchronous theo mặc định. Spring Boot (Java) mạnh cho enterprise nhưng verbosity cao. Lý do chọn Node.js: (1) Tái sử dụng kỹ năng JavaScript giữa frontend và backend; (2) Hệ sinh thái npm phong phú; (3) Tích hợp tự nhiên với các thư viện khác trong stack (Bull, Mongoose, Passport.js).

Express.js expressjs2024 là framework web minimalist cho Node.js, cung cấp routing, middleware pipeline và utility functions. Kiến trúc module trong đồ án tuân theo pattern: mỗi domain (auth, rovs, trips, dives, media...) có `model.js`, `service.js`, `controller.js` và `routes.js` riêng, đảm bảo tách biệt quan tâm rõ ràng.

3.2.2 MongoDB và Mongoose

MongoDB mongodb2024 là hệ quản trị cơ sở dữ liệu NoSQL document-oriented, lưu trữ dữ liệu dưới dạng BSON (Binary JSON). Đặc điểm quan trọng: schema flexible cho phép các document trong cùng collection có số trường khác nhau — phù hợp với sensor data của ROV, vì các model ROV khác nhau có thể có bộ cảm biến khác nhau.

So với PostgreSQL: Relational database phù hợp hơn khi cần ACID transactions phức tạp và queries với nhiều JOIN. Với hệ thống quản lý ROV, phần lớn queries đơn giản (get trip by ID, list dives of trip), không yêu cầu transaction phức tạp. MongoDB Atlas cung cấp free tier 512MB đủ cho demo, với geospatial indexing hỗ trợ queries theo GPS.

Mongoose mongoose2024 là ODM (Object Document Mapper) cho MongoDB trên Node.js, cung cấp schema validation, middleware hooks và query builder. Sử dụng Mongoose để đảm bảo data integrity ở tầng application.

3.2.3 Redis và Bull Queue

Redis redis2024 là in-memory data structure store hỗ trợ nhiều kiểu dữ liệu (string, hash, list, set, sorted set). Trong hệ thống, Redis được dùng cho

hai mục đích: (1) Token blacklist: khi người dùng logout, refresh token được lưu vào Redis với TTL tương ứng, ngăn tái sử dụng; (2) Backend storage cho Bull Queue.

Bull là thư viện message queue cho Node.js dựa trên Redis. Được dùng để xử lý bất đồng bộ các tác vụ tốn thời gian: AI object detection (YOLO), AI trip summary (Gemini) và email notification. Bull hỗ trợ retry tự động, delay jobs và priority queue. So với RabbitMQ: Bull đơn giản hơn trong setup (không cần cài server riêng vì dùng Redis đã có), phù hợp với triển khai single-instance trên VPS.

3.2.4 AWS S3 và presigned URL

AWS S3 (Simple Storage Service) là dịch vụ object storage của Amazon, cho phép lưu trữ file với độ tin cậy cao và khả năng mở rộng không giới hạn. Điểm kỹ thuật quan trọng: hệ thống sử dụng *presigned URL* — URL có chữ ký tạm thời (5 phút) cho phép client upload trực tiếp lên S3 mà không cần route file qua backend server. Điều này giảm đáng kể băng thông và tải CPU của server khi xử lý file video lớn.

So với Google Cloud Storage hay Cloudinary: S3 là lựa chọn phổ biến nhất với hệ sinh thái phong phú nhất (SDK Node.js, Lambda integration). Cloudinary có nhiều tính năng xử lý media built-in nhưng chi phí cao hơn.

3.3 Xác thực và bảo mật

3.3.1 JSON Web Token (JWT)

JWT **jwt_rfc** là tiêu chuẩn mở (RFC 7519) định nghĩa cách truyền thông tin an toàn giữa các bên dưới dạng JSON object được ký. Một JWT gồm ba phần: Header (thuật toán ký), Payload (claims: userId, role, exp), và Signature. Server ký bằng secret key (HMAC-SHA256) — bất kỳ thay đổi nào trong payload sẽ làm signature không hợp lệ.

Hệ thống dùng mô hình token hai tầng: access token (15 phút, lưu memory) và refresh token (7 ngày, lưu HttpOnly cookie). Khi access token hết hạn, axios interceptor tự động gọi `POST /auth/refresh` để lấy access token mới, trong suốt với người dùng.

3.3.2 Google OAuth 2.0

Google OAuth 2.0 cho phép đăng nhập bằng tài khoản Google mà không cần tạo mật khẩu mới. Được triển khai qua **Passport.js** với Google Strategy: khi người dùng chọn “Continue with Google”, họ được redirect đến Google consent screen; sau khi đồng ý, Google callback về back-

end với auth code; backend đổi code lấy profile, tìm/tạo user trong DB và trả JWT. Người dùng Google không có password trong DB (field `password` optional), và tab “Change Password” bị ẩn trong profile.

3.4 Trí tuệ nhân tạo và học máy

3.4.1 YOLOv8 — Object Detection

YOLO (You Only Look Once) **redmon2016yolo** là họ mô hình object detection tiên phong theo hướng one-stage detection: thay vì hai bước (region proposal + classification), YOLO giải quyết object detection như bài toán regression duy nhất trên toàn bộ ảnh trong một lần forward pass.

YOLOv8 jocher2023yolov8 là phiên bản mới nhất của Ultralytics, cải tiến kiến trúc C2f (Cross Stage Partial network), anchor-free detection head và NMS tốc độ cao. Mô hình nano (YOLOv8n) có kích thước chỉ khoảng 6MB, cho phép inference trên CPU trong khoảng 100–500ms/frame — phù hợp với triển khai trên VPS 4GB RAM mà không cần GPU.

Được triển khai trong microservice Python FastAPI, xử lý cả ảnh (predict) và video (track với ByteTracker, adaptive frame sampling: 0.2s/frame cho video ngắn hơn 30 giây, 1.0s/frame cho video dài hơn 3 phút).

So với Detectron2: Detectron2 mạnh hơn nhưng nặng hơn nhiều và khó deploy. Custom CNN từ đầu không phù hợp vì thiếu dữ liệu training có nhãn.

3.4.2 Google Gemini 2.5 Flash — Text Generation

Google Gemini 2.5 Flash gemini2024 là LLM (Large Language Model) đa phương thức của Google DeepMind, nổi bật với tốc độ inference cao và context window lớn (1 triệu token). Được sử dụng để sinh tóm tắt trip tự động từ structured prompt chứa: tên trip, thời gian, địa điểm (lấy từ Nominatim reverse geocoding), danh sách dives với trạng thái và thông số sensor.

So với GPT-4 (OpenAI) hay Claude (Anthropic): Gemini được chọn vì free tier Google AI Studio (1500 request/ngày) phù hợp với quy mô demo, và tích hợp dễ dàng qua Python SDK.

3.4.3 Z-Score — Phát hiện bất thường

Z-Score là phương pháp thống kê đo lường khoảng cách của một điểm dữ liệu so với giá trị trung bình, tính theo đơn vị độ lệch chuẩn **zscore_anomaly**:

$$z = \frac{x - \mu}{\sigma} \quad (3.1)$$

trong đó x là giá trị đo được, μ là giá trị trung bình và σ là độ lệch chuẩn của tập dữ liệu. Điểm được đánh dấu là bất thường khi $|z| > 2,5$ — tức là giá trị lệch hơn 2,5 lần độ lệch chuẩn so với trung bình.

Phương pháp Z-Score được chọn vì: (1) Độ phức tạp $O(n)$ cho phép tính realtime khi query; (2) Không cần training data — chỉ cần giả định phân phối xấp xỉ chuẩn của sensor data; (3) Kết quả dễ diễn giải (giá trị z cụ thể cho người vận hành biết mức độ bất thường).

So với IQR (Interquartile Range): IQR robust hơn với outlier nhưng khó giải thích bằng số cụ thể. Isolation Forest mạnh hơn cho bất thường đa chiều nhưng cần fitting và không phù hợp với streaming sensor data.

3.4.4 Nominatim — Reverse Geocoding

Nominatim **nominatim2024** là search engine cho dữ liệu OpenStreetMap, cung cấp API reverse geocoding miễn phí: từ tọa độ GPS (lat, lng) trả về tên địa điểm con người đọc được. Được sử dụng khi operator upload sensor CSV có lat/lng — hệ thống tự động gọi Nominatim để lưu `dive.locationName`, sau đó dùng tên địa điểm này trong prompt AI trip summary thay vì tọa độ số.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày thiết kế kiến trúc, thiết kế chi tiết cơ sở dữ liệu và API, quy trình xây dựng ứng dụng, minh họa các chức năng thông qua giao diện thực tế, kết quả kiểm thử và mô hình triển khai.

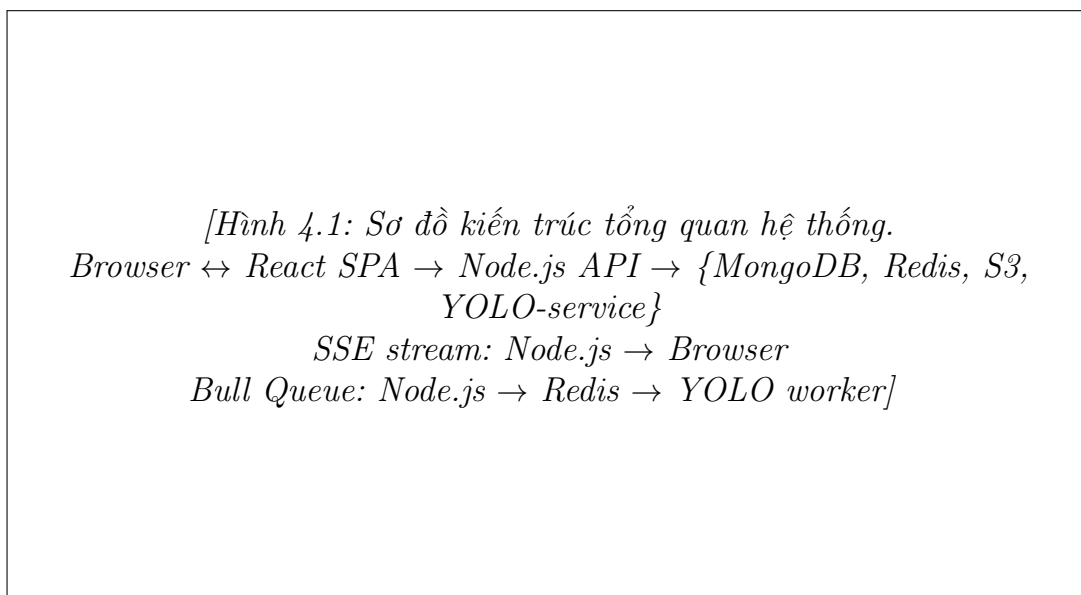
4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống áp dụng kiến trúc ba tầng (Three-Tier Architecture) kết hợp với microservice tách biệt cho tác vụ AI:

- **Tầng trình bày (Presentation Tier):** Ứng dụng React SPA chạy trên trình duyệt, giao tiếp với backend qua HTTP/REST và nhận push notification qua SSE.
- **Tầng ứng dụng (Application Tier):** Node.js/Express REST API xử lý business logic, xác thực, phân quyền, và điều phối các tác vụ bất đồng bộ qua Bull Queue.
- **Tầng dữ liệu (Data Tier):** MongoDB Atlas (persistent storage), Redis (cache + queue), AWS S3 (file storage).

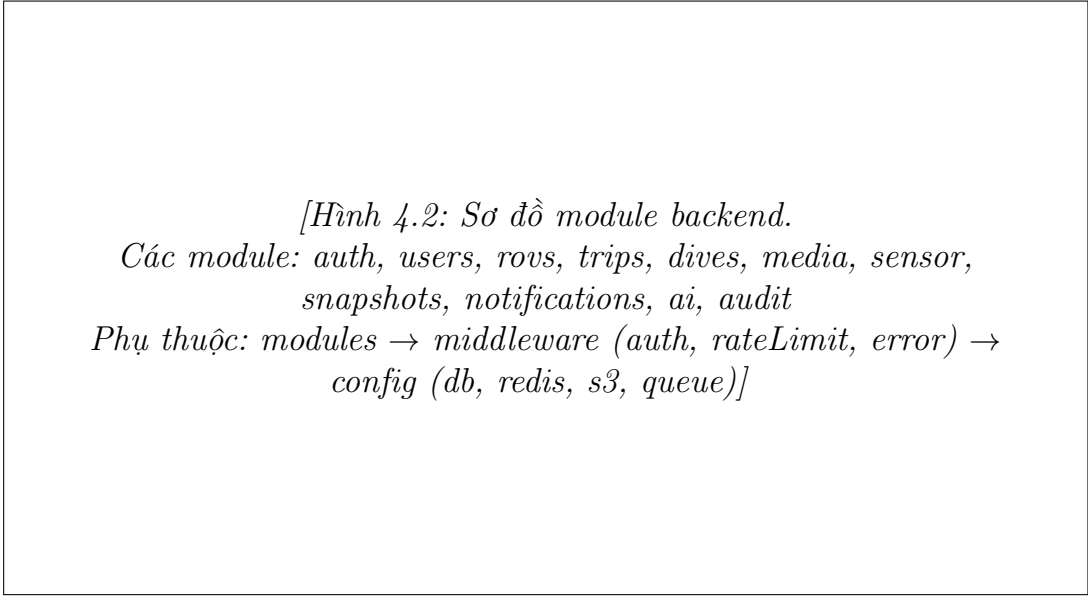
Ngoài ba tầng chính, một microservice Python FastAPI độc lập đảm nhận toàn bộ tác vụ AI (YOLOv8 inference). Microservice này giao tiếp với backend qua HTTP nội bộ trong Docker network, nhận job từ Bull Queue.



Hình 4.1: Kiến trúc tổng quan hệ thống quản lý ROV

4.1.2 Thiết kế tổng quan module

Backend được tổ chức theo mô hình module, mỗi module tương ứng với một domain nghiệp vụ:



*[Hình 4.2: Sơ đồ module backend.
Các module: auth, users, rovs, trips, dives, media, sensor,
snapshots, notifications, ai, audit
Phụ thuộc: modules → middleware (auth, rateLimit, error) →
config (db, redis, s3, queue)]*

Hình 4.2: Sơ đồ module backend Node.js

Mỗi module tuân theo cấu trúc bốn tầng: `model.js` (Mongoose schema), `service.js` (business logic), `controller.js` (HTTP handler), `routes.js` (Express router). Nguyên tắc thiết kế quan trọng: controller không chứa business logic, chỉ validate input và gọi service; service không phụ thuộc Express request/response, có thể test độc lập.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB với chín collection chính, có quan hệ reference (ObjectId) giữa các document:

*[Hình 4.3: Sơ đồ ER (Entity-Relationship) MongoDB.
User $\leftarrow$ ROV $\leftarrow$ Trip $\leftarrow$ Dive $\leftarrow$ SensorData
Dive $\leftarrow$ Media $\leftarrow$ Snapshot
User $\leftarrow$ Notification
User $\leftarrow$ AuditLog]*

Hình 4.3: Sơ đồ quan hệ giữa các collection MongoDB

Bảng ?? mô tả các trường chính của từng collection:

Bảng 4.1: Mô tả schema các collection chính

Collection	Trường chính
User	email, fullName, role, googleId, status
ROV	name, model, serialNumber, status
Trip	name, rovid, status, startTime, endTime, aiSummary
Dive	tripId, name, status, gpsLocation, locationName, sensorCount
SensorData	diveId, timestamp, depth, temp, pressure, lat, lng, yaw, pitch, roll, voltage
Media	diveId, s3Key, url, type, size, status, order, labels, analysisStatus, recorded
Snapshot	diveId, type, parentMediaId, imageS3Key, startTime, endTime, aiLabels

4.2.2 Thiết kế API RESTful

API tuân theo chuẩn RESTful với prefix `/api/v1`. Bảng ?? liệt kê các endpoint nhóm chính:

Bảng 4.2: Các endpoint API chính (nhóm theo resource)

Method	Endpoint	Mô tả
POST	<code>/auth/login</code>	Đăng nhập, trả JWT
POST	<code>/auth/refresh</code>	Làm mới access token
PATCH	<code>/auth/me</code>	Cập nhật thông tin cá nhân
GET	<code>/rovs</code>	Danh sách ROV (phân trang)
POST	<code>/rovs</code>	Tạo ROV mới
PATCH	<code>/rovs/:id</code>	Cập nhật ROV
GET	<code>/trips</code>	Danh sách trip (filter, search)
POST	<code>/trips</code>	Tạo trip mới
POST	<code>/trips/:id/ai-summary</code>	Enqueue AI summary job
GET	<code>/trips/:tripId/dives</code>	Danh sách dive của trip
POST	<code>/trips/:tripId/dives</code>	Tạo dive
POST	<code>/dives/:id/sensor-data/upload</code>	Upload sensor CSV
GET	<code>/dives/:id/sensor-data</code>	Lấy sensor data + stats + anomalies
GET	<code>/media/dive/:diveId</code>	Media của dive
GET	<code>/media/presigned-url</code>	Lấy presigned URL upload S3
POST	<code>/media/:id/analyze</code>	Enqueue YOLO job
GET	<code>/notifications/stream</code>	SSE endpoint
GET	<code>/notifications</code>	Danh sách thông báo
GET	<code>/audit</code>	Audit log (admin only)

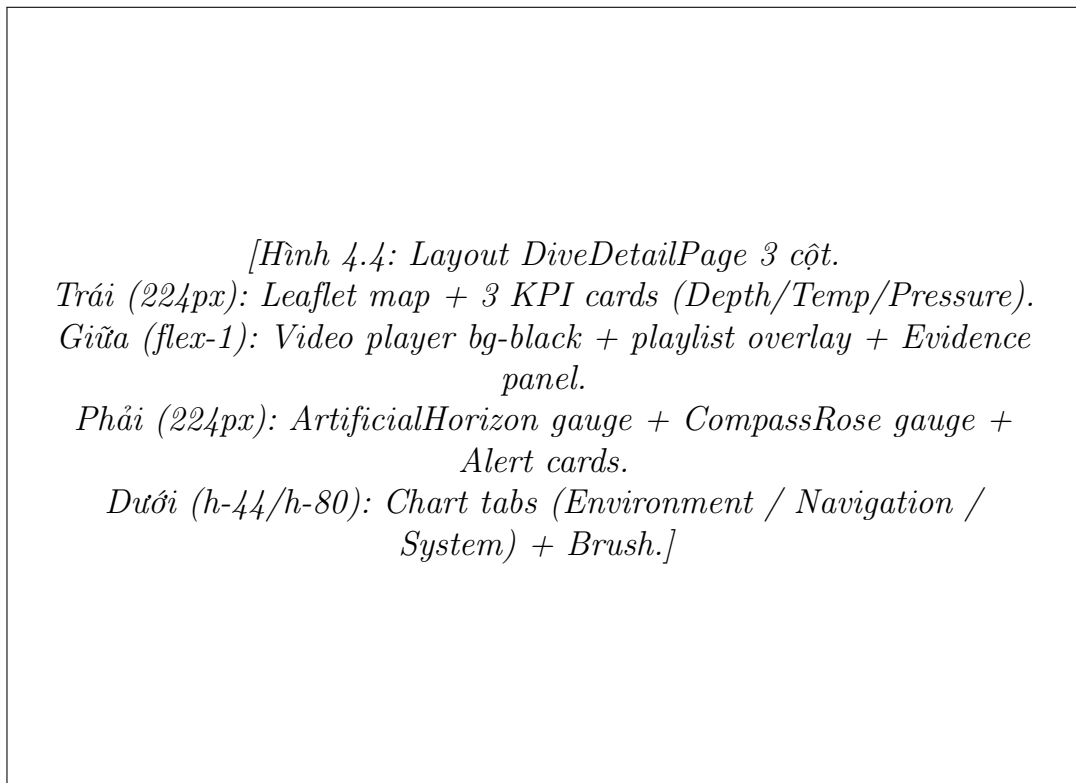
Tất cả response tuân theo format chuẩn: `{"success": true, "message": "...", "data": ...}`. Paginated response có cấu trúc `data: {data: [...], total, page, totalPages}`.

4.2.3 Thiết kế giao diện người dùng

Giao diện được thiết kế theo nguyên tắc *design token system*: toàn bộ màu sắc, spacing và typography được định nghĩa qua CSS variables trong `:root` (light mode) và `.dark` (dark mode). Tailwind CSS đọc các biến này qua hàm `rgb(var(-variable))`, đảm bảo chuyển đổi dark/light mode không cần viết CSS riêng.

Màn hình target chính: 1920×1080px (desktop operator). Responsive break-

DiveDetailPage Cockpit Layout là màn hình phức tạp nhất, được thiết kế dạng “cockpit” ba cột không cuộn, fit 100vh trừ navbar:



Hình 4.4: Layout cockpit của trang DiveDetailPage

4.2.4 Biểu đồ trình tự — Luồng upload sensor data

[Hình 4.5: Sequence diagram luồng Upload Sensor Data.
Operator → Frontend: Chọn file CSV
Frontend → Frontend: Parse & validate
Frontend → Backend: POST /dives/:id/sensor-data/upload (array
readings)
Backend → MongoDB: insertMany(SensorData)
Backend → Nominatim API: reverse geocode (nếu có GPS)
Backend → MongoDB: update dive.locationName
Backend → Frontend: 201 + {count, locationName}
Frontend: Cập nhật badge sensor count]

Hình 4.5: Biểu đồ trình tự luồng upload sensor data

4.3 Xây dựng ứng dụng

4.3.1 Công cụ và thư viện sử dụng

Bảng 4.3: Danh sách công cụ và thư viện chính

Mục đích	Công cụ / Thư viện	Phiên bản
Frontend framework	React	18.3
Build tool	Vite	5.x
CSS framework	Tailwind CSS	3.x
HTTP client	Axios	1.x
Server state	TanStack Query	5.x
Client state	Zustand	4.x
Charts	Recharts	2.x
Map	Leaflet.js	1.9
UI components	shadcn/ui	latest
Backend runtime	Node.js	20 LTS
Web framework	Express.js	4.x
Database ODM	Mongoose	8.x
Auth	Passport.js + JWT	0.7 / 9.x
Queue	Bull	4.x
Cache	ioredis	5.x
S3 SDK	@aws-sdk/client-s3	3.x
AI client	@google/generative-ai	0.x
AI microservice	Python	3.11
API framework	FastAPI	0.111
Object detection	Ultralytics YOLOv8	8.x
Database	MongoDB Atlas	7.0
Cache/Queue	Redis	7.x
Reverse proxy	Nginx	1.25
Container	Docker + Compose	25.x

4.3.2 Kết quả đạt được

Bảng 4.4: Thống kê mã nguồn và tính năng đạt được

Thống kê	Giá trị
Số dòng code frontend (JSX/JS)	~18.000
Số dòng code backend (JS)	~6.500
Số dòng code YOLO service (Python)	~350
Số module backend	11
Số component React	~65
Số API endpoint	42
Số collection MongoDB	9
Số chức năng hoàn thành	40/40

4.3.3 Minh họa các chức năng chính

Dashboard tổng quan: Hiển thị 4 stat cards (Total Trips, Running Dives, Active ROVs, Total Users), biểu đồ donut phân bố trip theo trạng thái, biểu đồ bar dives theo trạng thái, biểu đồ horizontal bar ROV utilization và biểu đồ timeline 6 tháng.

[Hình 4.6: Screenshot Dashboard tổng quan — stat cards và biểu đồ phân tích]

Hình 4.6: Giao diện Dashboard tổng quan

Trang quản lý Trip: Danh sách trip với filter nâng cao (status, ROV, date range, search), badge màu theo trạng thái (planned: xám, running: xanh, completed: xanh lá, cancelled: đỏ), và export CSV/PDF.

[Hình 4.7: Screenshot trang quản lý Trip — danh sách với filter và export]

Hình 4.7: Giao diện trang quản lý Trip

DiveDetailPage — Cockpit View: Layout ba cột không cuộn với video player trung tâm, biểu đồ sensor bên dưới, bản đồ Leaflet và KPI cards bên trái, gauge hướng và cảnh báo bên phải.

[Hình 4.8: Screenshot DiveDetailPage cockpit — video player, chart sensor, bản đồ GPS, gauge ArtificialHorizon + CompassRose, alert cards]

Hình 4.8: Giao diện DiveDetailPage (Cockpit View)

Media Gallery với YOLO labels: Media được hiển thị trong gallery có thể drag-to-reorder. Mỗi media card hiển thị thumbnail, badge YOLO labels (màu xanh >80% confidence, màu vàng 50–80%), và spinner khi đang phân tích.

[Hình 4.9: Screenshot Media Gallery — YOLO labels badges, AIAalyzePopover]

Hình 4.9: Giao diện Media Gallery với YOLO detection labels

Trang Audit Log: Timeline hiển thị mọi hành động của admin/operator với timestamp, loại hành động, entity bị ảnh hưởng và người thực hiện.

[Hình 4.10: Screenshot Audit Log — timeline hành động với filter entity type]

Hình 4.10: Giao diện Audit Log

4.4 Kiểm thử

4.4.1 Phương pháp kiểm thử

Hệ thống được kiểm thử theo hai phương pháp: (1) **Kiểm thử hộp đen (Black-box testing)** — kiểm tra behavior của API và UI dựa trên input/output mà không xét implementation; (2) **Kiểm thử tích hợp (Integration testing)** — kiểm tra luồng end-to-end từ frontend qua backend đến database.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.4.2 Các trường hợp kiểm thử chức năng đăng nhập

Bảng 4.5: Kiểm thử chức năng đăng nhập

TC	Input	Expected	Actual	Kết quả
TC01	Email + mật khẩu hợp lệ	Redirect dashboard, JWT trả về	Đúng như expected	Pass
TC02	Mật khẩu sai	Thông báo lỗi, không chuyển trang	Hiển thị lỗi	Pass
TC03	Email không tồn tại	Thông báo lỗi “Invalid credentials”	Đúng (không tiết lộ email tồn tại hay không)	Pass
TC04	Tài khoản bị khóa (status=inactive)	Thông báo tài khoản bị vô hiệu hóa	Đúng như expected	Pass
TC05	Access token hết hạn, refresh hợp lệ	Tự động làm mới token, request tiếp tục	Interceptor hoạt động đúng	Pass
TC06	Đăng xuất, dùng lại refresh token cũ	401 Unauthorized (black-listed)	Token bị từ chối	Pass

4.4.3 Các trường hợp kiểm thử upload sensor data

Bảng 4.6: Kiểm thử chức năng upload sensor data

TC	Input	Expected	Kết quả
TC07	CSV hợp lệ, 60 readings, có GPS	Lưu 60 readings, dive.locationName cập nhật	Pass
TC08	CSV thiếu cột “depth” bắt buộc	Thông báo lỗi cột thiếu, không lưu	Pass
TC09	CSV có 2 spike bất thường ($z > 2.5$)	API trả anomalies array gồm 2 điểm	Pass
TC10	Upload lần 2 cho cùng dive	Data cũ bị xóa, lưu data mới	Pass
TC11	Viewer cố gắng upload	403 Forbidden	Pass

4.4.4 Kiểm thử pipeline YOLO

Bảng 4.7: Kiểm thử pipeline phân tích YOLO bất đồng bộ

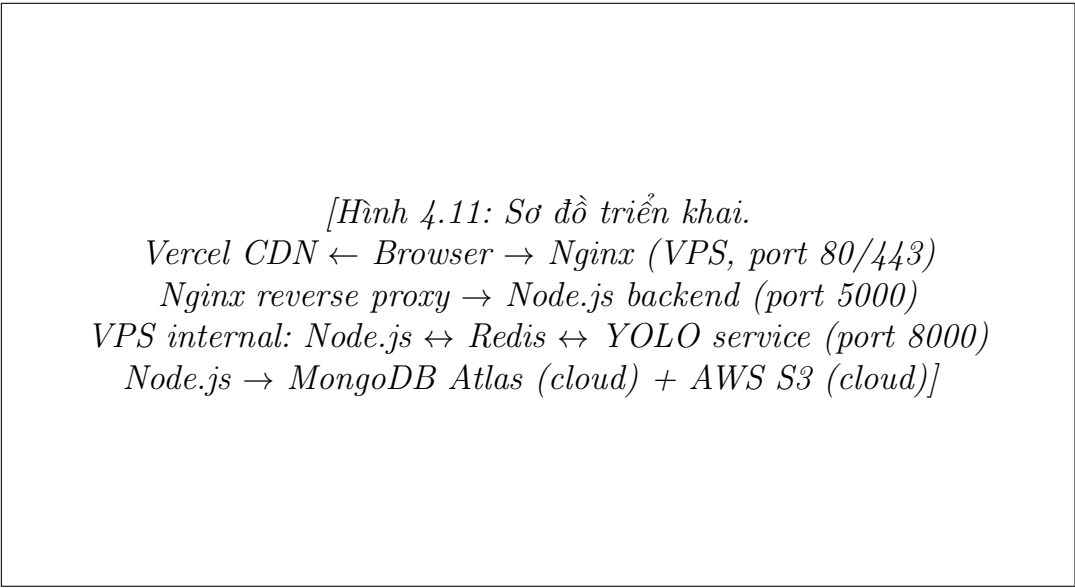
TC	Input	Expected	Kết quả
TC12	Ảnh JPG rõ ràng, trigger analyze	Labels hiển thị sau <5s, SSE push	Pass
TC13	Video 30 giây, trigger analyze	Labels hiển thị sau <30s, per-frame data	Pass
TC14	YOLO service down khi trigger	analysisStatus = “failed” sau 2 retry, không crash backend	Pass
TC15	Analyze với confidence=0.15 vs 0.5	Confidence thấp hơn detect nhiều labels hơn	Pass

Tổng kết kiểm thử: 15/15 test case pass. Không có case nào dẫn đến crash hệ thống hay data corruption.

4.5 Triển khai

4.5.1 Kiến trúc triển khai

Hệ thống được triển khai theo mô hình lai: frontend trên Vercel **vercel2024** (CDN toàn cầu, CI/CD tự động từ git push), backend stack trên VPS Contabo 4GB RAM:



Hình 4.11: Sơ đồ kiến trúc triển khai production

4.5.2 Docker Compose

Toàn bộ backend stack được quản lý bằng Docker Compose **docker2024** với bốn service:

```
1 services:
2   backend:
3     build: ./backend
4     env_file: .env
5     depends_on: [redis]
6     ports: ["5000:5000"]
7
8   yolo:
9     build: ./yolo-service
10    ports: ["8000:8000"]
11
12   redis:
13     image: redis:alpine
14     restart: always
15
16   nginx:
17     image: nginx:alpine
18     ports: ["80:80", "443:443"]
19     volumes: ["/etc/nginx/nginx.conf"]
```

Listing 4.1: Cấu trúc docker-compose.yml

Nginx **nginx2024** đảm nhiệm SSL termination, reverse proxy đến backend và phục vụ static file fallback. Triển khai production chỉ cần: **docker compose up -d**.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày năm giải pháp kỹ thuật nổi bật được phát triển trong đề án. Mỗi giải pháp được trình bày theo cấu trúc: (i) vấn đề/bài toán cần giải quyết, (ii) giải pháp được đề xuất và cách triển khai, (iii) kết quả đạt được.

5.1 Đồng bộ hóa dữ liệu cảm biến với video (Sensor-Video Sync)

5.1.1 Bài toán

Trong quy trình post-mission, operator upload dữ liệu cảm biến (file CSV) và video (file MP4) của cùng một lần lặn một cách độc lập. Dữ liệu cảm biến có timestamp tuyệt đối (UTC), trong khi video là file thô không mang thông tin thời gian bắt đầu quay. Kết quả là hai nguồn dữ liệu quan trọng này không thể đối chiếu lẫn nhau: khi xem video và thấy ROV tiếp xúc vật thể tại giây thứ 120, operator không thể biết nhiệt độ nước và độ sâu tại thời điểm đó là bao nhiêu.

5.1.2 Giải pháp

Giải pháp được đề xuất thêm trường `recordedAt: Date` (nullable) vào Media document. Trường này lưu timestamp UTC của *thời điểm bắt đầu quay* video. Khi operator upload video, họ có thể nhập thời điểm bắt đầu quay qua `datetime-local` input (tùy chọn). Khi đã có `recordedAt`, hệ thống tính `chartTimestamp` theo công thức:

$$t_{chart} = t_{recordedAt} + t_{video} \times 1000 \quad (5.1)$$

trong đó t_{video} là thời gian hiện tại của video tính bằng giây và $t_{recordedAt}$ là Unix timestamp tính bằng mili-giây. Giá trị t_{chart} được so khớp với trục X của biểu đồ sensor (cũng là Unix timestamp) để tìm điểm dữ liệu gần nhất.

Về mặt triển khai, frontend lắng nghe event `timeupdate` của video element, tính t_{chart} và truyền vào prop `referenceLineX` của component biểu đồ. `Recharts` render `<ReferenceLine x={referenceLineX} stroke="red" strokeDasharray="3 3"/>` — một đường dọc màu đỏ di chuyển realtime trên biểu đồ theo tiến trình video.

*[Hình 5.1: Minh họa Sensor-Video Sync.
Video đang phát tại giây 120 → ReferenceLine dọc hiển thị tại
timestamp tương ứng.
Tooltip chart hiển thị depth=15.2m, temp=24.1°C tại thời điểm đó.
Badge LIVE SYNC ở góc dưới trái video.]*

Hình 5.1: Minh họa tính năng Sensor-Video Sync

5.1.3 Kết quả

Tính năng cho phép operator xem lại video và đồng thời theo dõi tình trạng thiết bị tại từng thời điểm cụ thể. Khi phát hiện sự kiện bất thường trong video (ví dụ: ROV va chạm, ánh sáng thay đổi đột ngột), operator có thể ngay lập tức đối chiếu với dữ liệu sensor để xác định nguyên nhân. Biểu đồ hiển thị badge “LIVE SYNC” để phân biệt với chế độ xem độc lập.

5.2 Phát hiện bất thường cảm biến tự động (Z-Score Anomaly Detection)

5.2.1 Bài toán

Dữ liệu cảm biến ROV hoạt động dưới nước thường chứa các đột biến bất thường do: nhiễu điện từ khi ROV tiếp xúc kim loại, bong bóng nước che phủ cảm biến nhiệt độ, hoặc lỗi calibration tạm thời. Những đột biến này cần được phát hiện và hiển thị rõ ràng để kỹ sư có thể đánh giá tính hợp lệ của dữ liệu và phát hiện sự cố thiết bị thực sự.

5.2.2 Giải pháp

Hệ thống áp dụng Z-Score (đã giới thiệu tại mục ??) để phát hiện bất thường cho ba metric chính: depth, temp và pressure. Thuật toán được triển khai trong backend tại `sensor.service.js`:

```
1 function detectAnomalies(readings, metric, threshold =
  2.5) {
2   const values = readings.map(r => r[metric]).filter(v
    => v !== null);
3   const mean = values.reduce((a, b) => a + b, 0) /
```

```

        values.length;
4    const std = Math.sqrt(
5        values.map(v => Math.pow(v - mean, 2))
6            .reduce((a, b) => a + b, 0) / values.length
7    );
8    if (std === 0) return [];
9    return readings
10        .filter(r => r[metric] !== null && Math.abs((r[metric]
11            - mean) / std) > threshold)
12        .map(r => ({
13            index: readings.indexOf(r),
14            metric,
15            value: r[metric],
16            zScore: parseFloat(((r[metric] - mean) / std).
17                toFixed(2)),
18            timestamp: r.timestamp
19        }));

```

Listing 5.1: Thuật toán Z-Score detection trong sensor.service.js

API GET /dives/:id/sensor-data trả về cấu trúc:

```

1  {
2    "data": [...],
3    "stats": { "depth": { "min": 5.1, "max": 22.8, "avg":
4        14.3 }, ... },
5    "anomalies": [
6        { "index": 27, "metric": "depth", "value": 48.5,
7            "zScore": 3.12, "timestamp": "2026-05-07T08:27:00Z"
8        }
9    ]
10 }

```

Listing 5.2: Response format sensor data với anomalies

Phía frontend, anomaly được visualize bằng custom dot đỏ trên AreaChart của Recharts. Mỗi điểm trên biểu đồ so sánh index của nó với danh sách anomaly, nếu trùng thì render dot màu đỏ kích thước lớn hơn. Ngoài ra, panel “Anomalies Detected” bên phải cột liệt kê các điểm bất thường với giá trị z-score cụ thể và timestamp.

5.2.3 Kết quả

File test `sensor-sample-full.csv` (60 readings, GPS Đà Nẵng) được thiết kế với hai điểm đột biến nhân tạo: một spike depth=48.5m tại reading 27 ($z=3.12$) và một spike temp=42.1°C tại reading 41 ($z=3.47$). Hệ thống phát hiện và highlight chính xác cả hai điểm. Ngưỡng $|z| > 2,5$ được chọn để cân bằng giữa nhạy cảm (không bỏ sót bất thường thực) và specificity (không báo nhầm nhiều).

5.3 Pipeline phân tích YOLO bất đồng bộ với per-frame tracking

5.3.1 Bài toán

Object detection trên video ROV gặp hai thách thức kỹ thuật: (1) *Thời gian xử lý*: YOLOv8n inference mất 100–500ms/frame; video 3 phút với sampling 0.5s/frame có 360 frames, tổng thời gian 36–180 giây — không thể block HTTP request; (2) *Tracking liên tục*: Các detection của cùng một vật thể ở các frame khác nhau cần được liên kết lại (tracking), để khi operator click vào label trong video, video tự động seek đến frame có detection đó.

5.3.2 Giải pháp

Xử lý bất đồng bộ: Endpoint POST `/media/:id/analyze` không thực hiện inference trực tiếp. Thay vào đó: set `analysisStatus = 'pending'`, enqueue Bull job với `{mediaId, mimeType, model, confidence}`, và trả 202 Accepted ngay lập tức. Worker Bull lấy job từ Redis queue, download media từ S3 (qua presigned URL), gọi YOLO microservice:

```

1 mediaAnalysisQueue.process(async (job) => {
2   const { mediaId, model, confidence } = job.data;
3   const media = await Media.findById(mediaId);
4   const response = await axios.post(`${YOLO_URL}/detect`, {
5     mediaUrl: media.presignedUrl,
6     mediaType: media.mimeType,
7     model,
8     confidence
9   }, { timeout: 300000 }); // 5 phút cho video dài
10  await Media.findByIdAndUpdate(mediaId, {
11    labels: response.data,
12    analysisStatus: 'done'
13  });
14  await notificationService.create(userId, '
    media_analysis_done', ...);

```

15 });

Listing 5.3: Bull worker gọi YOLO service

Per-frame tracking: Python microservice sử dụng `model.track(persist=True, tracker="bytetrack.yaml")` thay vì `model.predict()` cho video. ByteTracker gán `trackId` ổn định cho mỗi vật thể xuyên suốt các frame. Mỗi detection được lưu với `frameTime` chính xác (giây trong video) và `trackId`. Adaptive sampling interval giảm chi phí tính toán: video ngắn hơn 30 giây sample mỗi 0.2s, video dài hơn 3 phút sample mỗi 1.0s.

Per-frame bbox overlay: Frontend filter labels theo `currentTime` của video với nearest-frame logic (tìm `frameTime` gần nhất, ẩn nếu gap >0.4s). SVG bbox overlay được căn chỉnh chính xác với `object-fit: contain` của video.

5.3.3 Kết quả

Pipeline hoạt động hoàn toàn bất đồng bộ: operator kích hoạt analyze và có thể tiếp tục làm việc; khi YOLO xử lý xong, thông báo SSE đẩy về trình duyệt và labels tự động hiển thị mà không cần reload trang. Operator có thể điều chỉnh model và confidence ngưỡng qua `AIAnalyzePopover` trước khi chạy, phù hợp với đặc thù môi trường (nước đục cần confidence thấp hơn).

5.4 Hệ thống Evidence — Bằng chứng trực quan từ video

5.4.1 Bài toán

Khi xem lại video ROV, operator thường cần đánh dấu các sự kiện quan trọng: phát hiện vật thể lạ, vết nứt trên kết cấu, sinh vật biển hiếm gặp. Phân tích YOLO toàn bộ video dài có thể mất hàng phút; ngoài ra, nhiều sự kiện cần ghi chú bổ sung từ operator mà YOLO không thể cung cấp. Cần một cơ chế cho phép đánh dấu nhanh từng sự kiện cụ thể trong khi xem video.

5.4.2 Giải pháp

Hệ thống Evidence cung cấp hai công cụ đánh dấu:

Photo Snapshot: Khi operator bấm nút trong lúc video đang phát, frontend dùng Canvas API để capture frame hiện tại thành PNG, gửi lên backend qua base64 (body limit 8MB). Backend upload PNG lên S3, tạo document Snapshot với `type = 'photo'`, `imageS3Key`, `imageTime` (giây trong video tại thời điểm capture). Nếu bbox overlay YOLO đang hiển thị, các bbox được “burn-in” vào ảnh canvas trước khi capture.

Video Clip: Operator bấm để bắt đầu đánh dấu (`clipStart`), bấm lại để

kết thúc (`clipEnd`). Backend tạo Snapshot với `type = 'clip'`, `startTime`, `endTime`, và thumbnail PNG của frame đầu. Clip không copy video — chỉ lưu metadata thời gian; khi xem lại, EvidenceViewer phát cùng video gốc nhưng constrain `currentTime` trong range `[startTime, endTime]`.

Phân tích YOLO riêng cho evidence: `POST /snapshots/:id/analyze` gọi YOLO service chỉ trên range clip (truyền `startTime/endTime` vào `DetectRequest`), tiết kiệm đáng kể thời gian so với phân tích toàn bộ video.

5.4.3 Kết quả

Evidence system cho phép operator tạo bộ sưu tập bằng chứng có chú thích trực tiếp từ video mà không cần công cụ ngoài. Mỗi evidence có YOLO labels riêng, bbox overlay và có thể thêm ghi chú. Collection evidence của một dive cung cấp “highlight reel” các sự kiện quan trọng, hỗ trợ viết báo cáo chuyển lặn nhanh hơn. Evidence được tách biệt hoàn toàn khỏi Media playlist (không xuất hiện trong gallery chính).

5.5 Tổng hợp trip tự động với AI (Gemini Trip Summary)

5.5.1 Bài toán

Sau mỗi chuyến hoạt động, operator phải viết báo cáo tổng hợp thủ công: thống kê số lần lặn, kết quả từng lần, thông số đáng chú ý, tình trạng thiết bị. Việc này lặp đi lặp lại theo khuôn mẫu nhất định nhưng vẫn tốn 30–60 phút mỗi chuyến và dễ bị thiếu thông tin nếu operator không nhớ đủ chi tiết.

5.5.2 Giải pháp

Hệ thống tích hợp Google Gemini 2.5 Flash để sinh tóm tắt trip tự động. Quy trình hoàn toàn bất đồng bộ qua Bull Queue:

Structured prompt engineering: Backend thu thập toàn bộ context của trip: tên, thời gian, địa điểm (từ `dive.locationName` do Nominatim cung cấp — tên con người đọc được thay vì tọa độ số), số lần lặn, trạng thái từng lần, số readings sensor, anomalies phát hiện được, số media và YOLO labels nổi bật. Tất cả được format thành structured JSON prompt:

```
1  const prompt = `
2  B n l k s p h n t c h d l i u ROV. H y
   v i  t b o c o t m t t
3  c h u y  n l n s a u b n g t i  n g V i t , n g n g n
   v c h u y n n g h i p (200-300 t ):
4
5  T H N G T I N C H U Y  N L N :
6  - T n : ${trip.name}
```



```

7 - ROV: ${trip.rovName}
8 - Thời gian: ${trip.startTime} - ${trip.endTime}
9 - a      im      : ${locationName || 'Kh ng c GPS'}
10
11 KHU N L N (${dives.length} l n ):
12 ${dives.map(d =>
13     '* ${d.name}: ${d.status},          s u TB ${d.avgDepth}
14     m, ' +
15     '${d.sensorCount} readings, ${d.anomalyCount} b t
16     t h Íng '
17 ).join('\n')}
18
19 V T T H P H T H I N (YOLO): ${topLabels.join(', ')}
20 || 'Ch a p h n t c h'}
21
22 Vi ết b o c o theo c u tr c: T ng quan K ết
23 q u t ng l n l n Ghi c h k t h u t .
24 ' ;
    
```

Listing 5.4: Cấu trúc prompt AI trip summary

Async flow: POST /trips/:id/ai-summary set aiSummary.status = 'pending', enqueue job, trả 202. Bull worker gọi Gemini API, lưu kết quả vào aiSummary.content với status = 'done'. SSE push thông báo ai_summary_done đến operator, frontend invalidate React Query cache và hiển thị summary.

5.5.3 Kết quả

Tính năng sinh báo cáo trong khoảng 5–15 giây tùy độ phức tạp của trip. Báo cáo tạo ra có cấu trúc nhất quán, đề cập đầy đủ các lần lặn, nêu bật các bất thường sensor và YOLO labels. Operator vẫn có thể chỉnh sửa bằng tay sau khi AI tạo xong, hoặc nhấn “Regenerate” với cùng dữ liệu nếu kết quả chưa ưng ý. Tính năng chỉ available khi trip có status = 'completed' và chỉ operator/admin mới thấy button.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã hoàn thành việc thiết kế và xây dựng hệ thống web quản lý và giám sát ROV tích hợp trí tuệ nhân tạo, đáp ứng toàn bộ 40 yêu cầu chức năng và phi chức năng đề ra ban đầu.

Kết quả đạt được so với mục tiêu ban đầu:

Bảng 6.1: Đối chiếu kết quả với mục tiêu đề ra

Mục tiêu	Kết quả
Quản lý fleet ROV, trip và dive với RBAC 3 cấp	Hoàn thành
Upload và visualize dữ liệu sensor từ CSV	Hoàn thành
Phát hiện bất thường tự động (Z-Score)	Hoàn thành
Quản lý media trên AWS S3	Hoàn thành
Phân tích object detection với YOLOv8 (async)	Hoàn thành
Đồng bộ sensor-video theo thời gian thực	Hoàn thành
Hệ thống Evidence (snapshot + clip)	Hoàn thành
AI trip summary với Gemini 2.5 Flash	Hoàn thành
Thông báo realtime qua SSE	Hoàn thành
Giao diện responsive, dark/light mode	Hoàn thành
Triển khai Docker Compose trên VPS	Hoàn thành

So sánh với các giải pháp hiện có: Hệ thống vượt trội so với QGround-Control và BlueOS ở khả năng quản lý fleet, lưu trữ và phân tích dữ liệu post-mission. Điểm khác biệt quan trọng nhất so với cả các hệ thống thương mại là tích hợp AI hai tầng: YOLOv8 cho computer vision và Gemini cho text generation, cùng với cơ chế Sensor-Video Sync độc đáo không thấy trong các giải pháp hiện có.

Kinh nghiệm rút ra: Kiến trúc microservice cho AI (Python FastAPI tách biệt) là lựa chọn đúng đắn — cho phép swap model YOLO mà không ảnh hưởng backend Node.js, và có thể scale CPU riêng biệt. Bull Queue + SSE tạo ra trải nghiệm người dùng tốt cho tác vụ AI nặng: không block request, không cần polling, thông báo tức thì khi xong.

Hạn chế: Hệ thống hiện chạy single-instance, chưa có cơ chế horizontal scaling. Token blacklist và Bull Queue dùng cùng Redis instance — có thể tách

ra trong production lớn. YOLO model hiện chỉ là model general (YOLOv8n trained trên COCO), chưa fine-tune cho ảnh dưới nước ROV.

6.2 Hướng phát triển

Fine-tune YOLO cho ROV: Thu thập và gán nhãn dataset ảnh từ camera ROV (rạn san hô, mảnh vỡ tàu đắm, vết nứt đường ống, sinh vật biển đặc thù) để fine-tune YOLOv8m hoặc YOLOv8l. Model tùy biến sẽ có accuracy cao hơn nhiều so với model general trên ảnh dưới nước.

Multi-instance và horizontal scaling: Chuyển Bull Queue sang BullMQ với Redis Cluster, thêm load balancer trước nhiều Node.js instance. SSE connection lưu trong Redis Pub/Sub thay vì in-memory Map để share giữa các instance.

Mobile app cho Viewer: Xây dựng React Native app cho phép viewer xem báo cáo trip, ảnh và video trên điện thoại. Đặc biệt hữu ích cho khách hàng hoặc quản lý cấp cao cần xem báo cáo ngay sau khi nhận thông báo.

GCS realtime khi có kết nối: Khi tàu hoạt động gần bờ hoặc có satellite internet, tích hợp tự động upload sensor data realtime (websocket hoặc MQTT) thay vì chờ về bờ. Hệ thống có thể detect được hai mode: realtime (khi có kết nối) và post-mission (khi không có kết nối).

Dashboard phân tích xu hướng: Thêm các biểu đồ so sánh giữa nhiều trips và dives: xu hướng nhiệt độ nước theo tháng, ROV uptime statistics, YOLO detection frequency trends. Giúp kỹ sư phát hiện suy giảm hiệu suất ROV theo thời gian.

Tích hợp bathymetry map: Kết hợp GPS track từ nhiều dive với dữ liệu độ sâu để render bản đồ 3D đáy biển (bathymetric map) cho một trip hoàn chỉnh. Đây là tính năng có giá trị cao cho khảo sát địa chất biển.

PHỤ LỤC

A. CÀI ĐẶT VÀ CHẠY ỨNG DỤNG

A.1 Yêu cầu hệ thống

Môi trường development (local):

- Node.js 20 LTS
- Python 3.11
- Docker Desktop (Windows/Mac) hoặc Docker Engine (Linux)
- MongoDB Atlas account (free tier đủ dùng)
- AWS S3 bucket với IAM credentials
- Google AI Studio API key (Gemini)

A.2 Cài đặt và chạy local

Bước 1 — Clone repository:

```
1 git clone <repository-url>
2 cd rov-management
```

Listing A.1: Clone và cấu trúc thư mục

Bước 2 — Cấu hình biến môi trường backend:

Tạo file backend/.env với nội dung sau:

```
1 NODE_ENV=development
2 PORT=5000
3 MONGODB_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/rovd
4 JWT_SECRET=<random-secret-64-chars>
5 JWT_REFRESH_SECRET=<random-secret-64-chars>
6 AWS_ACCESS_KEY_ID=<aws-key>
7 AWS_SECRET_ACCESS_KEY=<aws-secret>
8 AWS_REGION=ap-southeast-1
9 S3_BUCKET=<bucket-name>
10 REDIS_URL=redis://localhost:6379
11 GEMINI_API_KEY=<google-ai-studio-key>
12 YOLO_SERVICE_URL=http://localhost:8000
13 CLIENT_URL=http://localhost:5173
```

Listing A.2: Nội dung file backend/.env

Bước 3 — Chạy Redis (cần cho Bull Queue và token blacklist):

```
1 docker run -d -p 6379:6379 redis:alpine
```

Bước 4 — Chạy YOLO microservice:

```
1 cd yolo-service
2 pip install -r requirements.txt
3 uvicorn main:app --host 0.0.0.0 --port 8000
```

Bước 5 — Chạy backend và frontend:

```
1 # Terminal 1 - Backend
2 cd backend && npm install && npm run dev
3
4 # Terminal 2 - Frontend
5 cd frontend && npm install && npm run dev
```

Truy cập <http://localhost:5173> để mở ứng dụng.

Bước 6 — Seed dữ liệu mẫu (tùy chọn):

```
1 cd backend && node src/scripts/seed.js
```

Tài khoản mặc định sau khi seed:

- admin@rov.local / Admin@123
- operator@rov.local / Operator@123
- viewer@rov.local / Viewer@123

A.3 Triển khai production với Docker Compose

```
1 # Tr n VPS - copy to n b source code
2 scp -r . user@vps-ip:/opt/rov-management
3
4 # T o .env production ( i CLIENT_URL v REDIS_URL
5 )
6 # CLIENT_URL=https://your-domain.vercel.app
7 # REDIS_URL=redis://redis:6379
8
9 # Build v c h y
10 cd /opt/rov-management
11 docker compose up -d --build
12
13 # Xem logs
14 docker compose logs -f backend
```

Listing A.3: Triển khai production

Frontend deploy lên Vercel: kết nối repository GitHub, set environment variable `VITE_API_URL=https://your-vps-domain.com/api/v1`.

A.4 Test data cho demo

File `test-data/sensor-sample-full.csv` cung cấp sẵn 60 readings với:

- GPS tọa độ Đà Nẵng (16.0544, 108.2022) ở dòng đầu
- Đủ 12 cột: timestamp, depth, temp, pressure, lat, lng, yaw, pitch, roll, voltage, battery_percent, humidity
- 2 spike bất thường nhân tạo (depth=48.5m tại reading 27 và temp=42.1°C tại reading 41) để demo Z-Score anomaly detection

B. ĐẶC TẢ USE CASE CHI TIẾT

Phụ lục này cung cấp đặc tả chi tiết bổ sung cho các use case quan trọng chưa được trình bày đầy đủ trong Chương 2.

B.1 UC07 — Quản lý người dùng (Admin)

Bảng B.1: Đặc tả use case UC07 — Quản lý người dùng

Tên use case	Quản lý người dùng
Tác nhân	Admin
Tiền điều kiện	Đã đăng nhập với role Admin
Hậu điều kiện	Role hoặc status người dùng được cập nhật
Luồng chính	1. Admin vào trang Users 2. Tìm kiếm/filter theo role 3. Chọn hành động: đổi role, kích hoạt, vô hiệu hóa 4. Hệ thống tự bảo vệ: không cho Admin tự đổi role hoặc vô hiệu hóa chính mình 5. Bulk operations: chọn nhiều user → đổi role hàng loạt
Luồng ngoại lệ	Admin cố thay đổi chính mình → Thông báo lỗi “Cannot modify own account”

B.2 UC08 — Xem audit log

Bảng B.2: Đặc tả use case UC08 — Xem audit log

Tên use case	Xem audit log
Tác nhân	Admin
Tiền điều kiện	Đã đăng nhập với role Admin
Hậu điều kiện	Không thay đổi trạng thái
Luồng chính	1. Admin vào trang Audit Log 2. Hệ thống hiển thị timeline các hành động theo thứ tự thời gian ngược 3. Mỗi entry hiển thị: avatar + tên user, action, entity type, entity ID, timestamp tương đối 4. Admin có thể filter theo entity type (trip, dive, user, rov)
Phạm vi ghi log	Hành động được ghi: tạo/xóa trip, tạo/xóa dive, xóa ROV, đổi role user, toggle status user, bulk operations, generate AI summary

B.3 UC09 — Tạo evidence từ video**Bảng B.3:** Đặc tả use case UC09 — Tạo evidence

Tên use case	Tạo evidence từ video
Tác nhân	Operator, Admin
Tiền điều kiện	Dive có ít nhất một media video đang được active trong DiveDetailPage
Hậu điều kiện	Snapshot document được tạo, ảnh/thumbnail lưu S3
Luồng chính — Photo	1. Video đang phát 2. Operator bấm nút 3. Canvas API capture frame hiện tại thành PNG 4. Upload base64 lên backend: POST /snapshots 5. Backend upload lên S3, tạo Snapshot document 6. Evidence panel cập nhật badge count
Luồng chính — Clip	1. Operator bấm → nút đổi sang đồ + animate-pulse 2. Video tiếp tục phát, hiển thị thời điểm bắt đầu 3. Operator bấm lại → backend tạo Snapshot clip với <code>startTime/endTime</code> 4. Backend capture thumbnail frame đầu clip, upload S3
Luồng ngoại lệ	Không có video active → Nút và không hiển thị

B.4 Định dạng file CSV cảm biến

File CSV phải có header dòng đầu với các cột sau:

Bảng B.4: Định dạng file CSV cảm biến ROV

Cột	Kiểu	Bắt buộc	Mô tả
timestamp	ISO 8601 DateTime	Có	Thời điểm đo
depth	float (m)	Có	Độ sâu
temp	float (°C)	Có	Nhiệt độ nước
pressure	float (bar)	Có	Áp suất
lat	float	Không	Vĩ độ GPS
lng	float	Không	Kinh độ GPS
yaw	float (°)	Không	Góc quay ngang
pitch	float (°)	Không	Góc ngả trước-sau
roll	float (°)	Không	Góc nghiêng trái-phải
voltage	float (V)	Không	Điện áp pin
battery_percent	float (%)	Không	Phần trăm pin còn lại
humidity	float (%)	Không	Độ ẩm buồng điện tử

Ví dụ một dòng hợp lệ:

```
1 2026-05-07T08:00:00Z
    ,10.5,24.3,1.23,16.0544,108.2022,183.2,-1.2,0.8,16.80,100,28.1
```

Listing B.1: Ví dụ một dòng CSV hợp lệ